If the parent object (the one declaring the `has_and_belongs_to_many` association) is unsaved (that is, `new_record?` returns `true`) then the child objects are not saved when they are added. All unsaved members of the association will automatically be saved when the parent is saved.
If you want to assign an object to a `has_and_belongs_to_many` association without saving the object, use the `collection.build` method.

4.5 Association Callbacks

Normal callbacks hook into the life cycle of Active Record objects, allowing you to work with those objects at various points. For example, you can use a `:before_save` callback to cause something to happen just before an object is saved.
Association callbacks are similar to normal callbacks, but they are triggered by events in the life cycle of a collection. There are four available association callbacks:

- `before_add`
- `after_add`
- `before_remove`
- `after_remove`

You define association callbacks by adding options to the association declaration. For example:

```
class Customer < ActiveRecord::Base
  has_many :orders, before_add: :check_credit_limit

  def check_credit_limit(order)
    ...
  end
end
```

Rails passes the object being added or removed to the callback.

You can stack callbacks on a single event by passing them as an array:

```
class Customer < ActiveRecord::Base
  has_many :orders,
    before_add: [:check_credit_limit, :calculate_shipping_charges]

  def check_credit_limit(order)
    ...
  end

  def calculate_shipping_charges(order)
    ...
  end
end
```

If a `before_add` callback throws an exception, the object does not get added to the collection. Similarly, if a `before_remove` callback throws an exception, the object does not get removed from the collection.

4.6 Association Extensions

You're not limited to the functionality that Rails automatically builds into association proxy objects. You can also extend these objects through anonymous modules, adding new finders, creators, or other methods. For example:

```
class Customer < ActiveRecord::Base
```